

Capitolo 4 - Crittografia Asimmetrica e Funzioni Hash

La funzione hash unidirezionale, o funzione hash sicura, è importante non solo nell'autenticazione dei messaggi ma anche nelle firme digitali. Tutte le funzioni hash operano secondo i seguenti principi generali: l'input (messaggio, file, ecc.) è visto come una sequenza di blocchi da n-bit; l'input viene elaborato un blocco alla volta in modo iterativo per produrre una funzione hash da n-bit.

Funzione HASH

Una delle funzioni hash più semplici è l'operazione **OR esclusivo (XOR)** bit per bit di ogni blocco. Questa può essere espressa come segue:

$$C_i = b_{i1} \oplus b_{i2} \oplus \dots \oplus b_{im}$$

dove:

- C_i = i-esimo bit del codice hash, $1 \leq i \leq n$,
- m = numero di blocchi da n bit nell'input,
- b_{ij} = i-esimo bit nel j-esimo blocco, e
- $\oplus$ = operazione XOR.

essa produce una parità semplice per ogni posizione di bit ed è nota come **controllo di ridondanza longitudinale (LRC)**. È ragionevolmente efficace per dati casuali come controllo di integrità dei dati.

	Bit 1	Bit 2	• • •	Bit n
Block 1	b_{11}	b_{21}		b_{n1}
Block 2	b_{12}	b_{22}		b_{n2}
	•	•	•	•
	•	•	•	•
	•	•	•	•
Block m	b_{1m}	b_{2m}		b_{nm}
Hash code	C_1	C_2		C_n

Un modo semplice per migliorare la situazione consiste nell'eseguire uno **scorrimento circolare di 1 bit**, o rotazione, sul valore hash dopo che ogni blocco è stato elaborato. La procedura può essere riassunta come segue:

1. Inizialmente impostare il valore hash a n bit a zero.
2. Elaborare ogni blocco successivo di dati a n bit come segue: a. Ruotare l'attuale valore hash a sinistra di 1 bit. b. Eseguire lo XOR del blocco nel valore hash.

Questo ha l'effetto di "randomizzare" l'input in modo più completo e superare le regolarità che appaiono nell'input. Sebbene la seconda procedura fornisca una buona misura dell'integrità dei dati, è virtualmente inutile per la sicurezza dei dati quando un codice hash crittografato viene utilizzato con un messaggio in chiaro. Dato un messaggio, è facile preparare un nuovo messaggio che produca quel codice hash: basta preparare il messaggio alternativo desiderato, quindi aggiungere un blocco da n bit che forzi il nuovo messaggio più il blocco a produrre il codice hash desiderato.

Sebbene un semplice XOR o uno XOR ruotato (RXOR) siano insufficienti se viene crittografato solo il codice hash, si potrebbe pensare che una funzione così semplice possa essere utile quando vengono crittografati sia il messaggio che il codice hash. Ma bisogna fare attenzione. Un'altra tecnica è quella

che utilizza XOR applicato a blocchi di 64 bit del messaggio e poi una crittografia dell'intero messaggio che utilizzava la modalità **CBC (Cipher Block Chaining)**. Possiamo definire lo schema come segue: dato un messaggio composto da una sequenza di blocchi a 64 bit $X_1, X_2, \dots, X_N$, definiamo il codice hash C come lo XOR blocco per blocco di tutti i blocchi e aggiungiamo il codice hash come blocco finale:

$$C = X_{N+1} = X_1 \oplus X_2 \oplus \dots \oplus X_N$$

Successivamente, si crittografa l'intero messaggio più il codice hash, utilizzando la modalità CBC per produrre il messaggio crittografato $Y_1, Y_2, \dots, Y_{N+1}$

$$\begin{aligned} X_1 &= IV \oplus D(K, Y_1) \\ X_i &= Y_{i-1} \oplus D(K, Y_i) \\ X_{N+1} &= Y_N \oplus D(K, Y_{N+1}) \end{aligned}$$

Ma X_{N+1} è il codice hash:

$$\begin{aligned} X_{N+1} &= X_1 \oplus X_2 \oplus \dots \oplus X_N \\ &= [IV \oplus D(K, Y_1)] \oplus [Y_1 \oplus D(K, Y_2)] \oplus \dots \oplus [Y_N \oplus D(K, Y_{N+1})] \end{aligned}$$

Poiché i termini nell'equazione precedente possono essere sottoposti a XOR in qualsiasi ordine, ne consegue che il codice hash non cambierebbe se i blocchi di testo cifrato venissero permutati.

La Funzione HASH sicura

Negli ultimi anni, la funzione hash più ampiamente utilizzata è stata il **Secure Hash Algorithm (SHA)**. Infatti, poiché quasi ogni altra funzione hash di largo consumo presentava sostanziali debolezze crittoanalitiche, lo SHA è rimasto l'unico algoritmo hash standardizzato fino al 2005.

	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512	SHA-512/224	SHA-512/256
Message size	< 2 ⁶⁴	< 2 ⁶⁴	< 2 ⁶⁴	< 2 ¹²⁸	< 2 ¹²⁸	< 2 ¹²⁸	< 2 ¹²⁸
Word size	32	32	32	64	64	64	64
Block size	512	512	512	1024	1024	1024	1024
Message digest size	160	224	256	384	512	224	256
Number of steps	80	64	64	80	80	80	80
Security	80	112	128	192	256	112	128

Sviluppato dal NIST e pubblicato come FIPS 180 nel 1993, lo SHA ha subito diverse revisioni:

- **SHA-0:** La versione originale (ritirata per debolezze).
- **SHA-1:** Revisione del 1995 (FIPS 180-1), produce un hash di 160 bit.
- **SHA-2:** Famiglia che comprende SHA-256, SHA-384 e SHA-512, introdotta nel 2002 (FIPS 180-2).
- **SHA-3:** Lo standard più recente (FIPS 202, 2015), basato su una struttura differente.

SHA-512

L'algoritmo accetta in input un messaggio con una lunghezza massima inferiore a 2¹²⁸ bit e produce come output un digest di 512 bit. Il messaggio viene elaborato in blocchi da 1024 bit.

I Passaggi dell'Algoritmo

- **Passaggio 1: Aggiunta dei bit di padding.** Il messaggio viene imbottito in modo che la sua lunghezza sia congruente a $896 \pmod{1024}$. Il padding consiste in un singolo bit "1" seguito dal numero necessario di bit "0".
- **Passaggio 2: Aggiunta della lunghezza.** Un blocco di 128 bit (trattato come intero big-endian) viene aggiunto al messaggio, contenente la lunghezza del messaggio originale prima del padding.
- **Passaggio 3: Inizializzazione del buffer hash.** Viene utilizzato un buffer di 512 bit per memorizzare i risultati intermedi e finali. Il buffer è rappresentato da otto registri da 64 bit (a, b, c, d, e, f, g, h), inizializzati ai seguenti valori esadecimali:

$a = 6A09E667F3BCC908$

$b = BB67AE8584CAA73B$

$c = 3C6EF372FE94F82B$

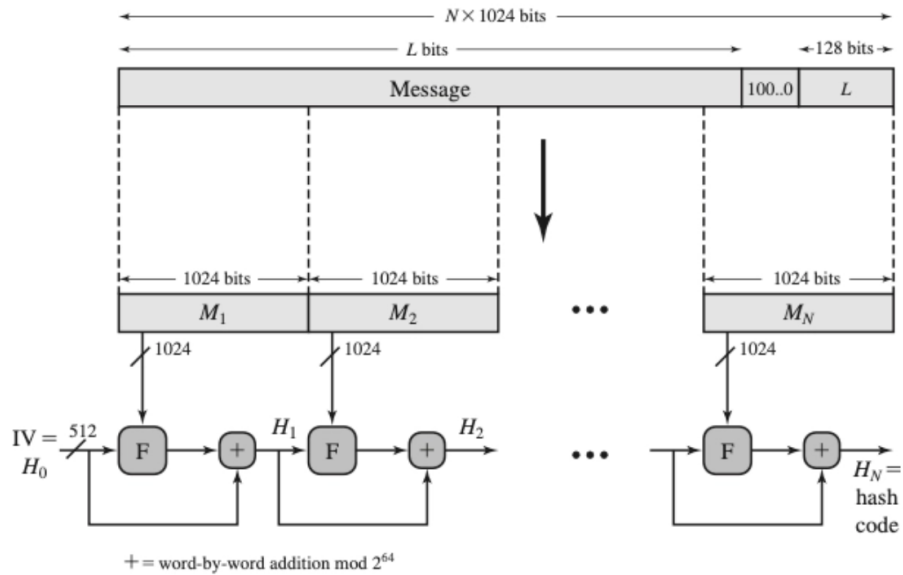
$d = A54FF53A5F1D36F1$

$e = 510E527FADE682D1$

$f = 9B05688C2B3E6C1F$

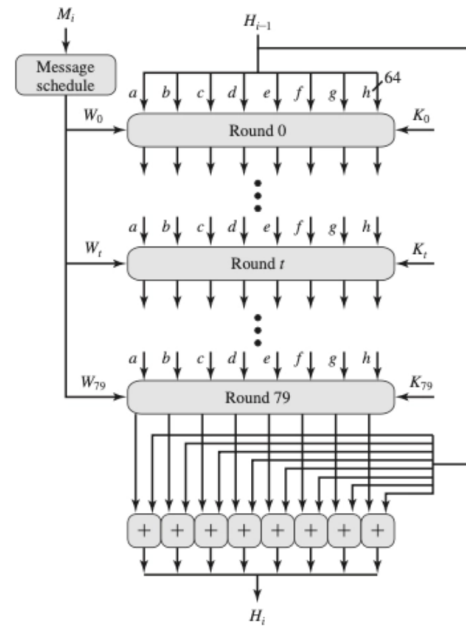
$g = 1F83D9ABFB41BD6B$

$h = 5BE0CD19137E2179$



- **Passaggio 4: Elaborazione in blocchi da 1024 bit.** Il cuore dell'algoritmo è un modulo composto da **80 round** (indicato come funzione F). Ogni round t prende come input il valore del buffer a 512 bit **abcdefgh** e lo aggiorna utilizzando:

1. Una word da 64 bit W_t , derivata dal blocco di dati corrente.
2. Una costante additiva K_t , dove $0 \leq t \leq 79$. Queste costanti eliminano regolarità nei dati di input.
3. Operazioni di scorrimento circolare e funzioni booleane primitive (AND, OR, NOT, XOR).



- **Passaggio 5: Output.** Dopo che tutti gli N blocchi da 1024 bit sono stati elaborati, l'output dello stadio N -esimo è il digest del messaggio a 512 bit.

SHA-512 possiede la proprietà per cui ogni bit del codice hash è funzione di ogni bit dell'input. La ripetizione complessa della funzione F produce risultati ben mischiati, rendendo estremamente difficile trovare due messaggi con lo stesso digest (resistenza alle collisioni).

SHA-3

Nonostante la sicurezza di SHA-2 sia ancora solida, il NIST ha indetto una competizione nel 2007 per SHA-3 per avere un'alternativa pronta qualora SHA-2 venisse compromesso. I requisiti per SHA-3 erano:

1. **Sostituzione immediata:** Deve poter sostituire SHA-2 nelle applicazioni esistenti (stesse lunghezze hash).
2. **Natura online:** Deve elaborare piccoli blocchi di dati senza richiedere l'intero messaggio in memoria.

SHA-3 è basato su una struttura a "spugna" (**Permutation-Based Hash**), sostanzialmente diversa da quella di SHA-1 e SHA-2. Questo garantisce che se una debolezza venisse scoperta in uno standard, l'altro rimarrebbe probabilmente sicuro.

HMAC

Negli ultimi anni, c'è stato un crescente interesse nello sviluppo di un MAC derivato da un codice hash crittografico, come lo SHA-1. Le motivazioni di questo interesse sono le seguenti:

- Le funzioni hash crittografiche generalmente vengono eseguite **più velocemente** via software rispetto agli algoritmi di cifratura convenzionali come il DES.
- Il codice di libreria per le funzioni hash crittografiche è **ampiamente disponibile**.

Una funzione hash come lo SHA-1 non è stata progettata per essere usata come MAC e non può essere utilizzata direttamente per tale scopo perché non si basa su una chiave segreta. Sono state avanzate diverse proposte per l'incorporazione di una chiave segreta in un algoritmo hash esistente. L'approccio che ha ricevuto il maggior supporto è **HMAC**.

Obiettivi di Progettazione di HMAC

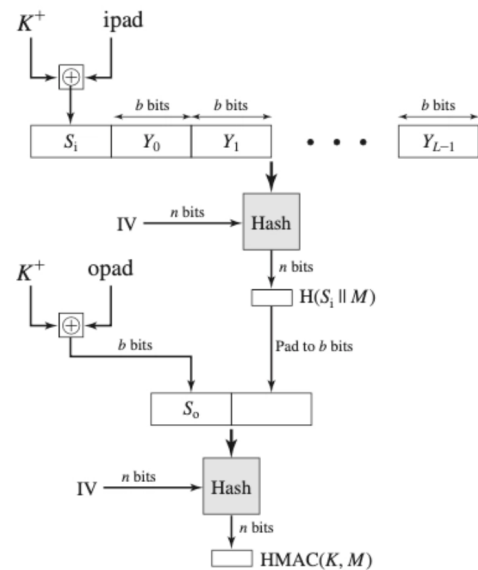
La RFC 2104 elenca i seguenti obiettivi di progettazione per HMAC:

- Utilizzare, senza modifiche, le funzioni hash disponibili — in particolare, funzioni hash che abbiano buone prestazioni in software e per le quali il codice sia liberamente e ampiamente disponibile.
- Consentire una **facile sostituibilità** della funzione hash integrata nel caso in cui vengano trovate o richieste funzioni hash più veloci o più sicure.
- Preservare le prestazioni originali della funzione hash senza subire un degrado significativo.
- Utilizzare e gestire le chiavi in modo semplice.
- Avere un'analisi crittografica ben compresa della robustezza del meccanismo di autenticazione basata su ipotesi ragionevoli sulla funzione hash integrata.

Algoritmo HMAC

La figura illustra il funzionamento generale di HMAC. Definiamo i seguenti termini:

- H = funzione hash integrata (es. SHA-1)
- M = messaggio di input a HMAC (incluso il padding specificato nella funzione hash integrata)
- Y_i = i -esimo blocco di M , $0 \leq i \leq (L - 1)$
- L = numero di blocchi in M
- b = numero di bit in un blocco
- n = lunghezza del codice hash prodotto dalla funzione hash integrata
- K = chiave segreta; se la lunghezza della chiave è maggiore di b , la chiave viene passata alla funzione hash per produrre una chiave a n bit; la lunghezza raccomandata è $\geq n$



- K^+ = K con l'aggiunta di zeri a sinistra in modo che il risultato sia lungo b bit
- ipad = **00110110** (36 in esadecimale) ripetuto $b/8$ volte
- opad = **01011100** (5C in esadecimale) ripetuto $b/8$ volte

Quindi HMAC può essere espresso come segue:

$$HMAC(K, M) = H[(K^+ \oplus \text{opad}) \parallel H[(K^+ \oplus \text{ipad}) \parallel M]]$$

In altre parole questo è il funzionamento:

1. **Aggiungere zeri** all'estremità sinistra di K per creare una stringa $K+$ di b bit (ad esempio, se K ha una lunghezza di 160 bit e $b=512$, a K verranno aggiunti 44 byte di zeri **0x00**).
2. Eseguire lo **XOR** (OR esclusivo bit a bit) di $K+$ con $ipad$ per produrre il blocco di b bit S_i .
3. Appendere M a S_i .
4. Applicare H allo stream generato nel passaggio 3.
5. Eseguire lo **XOR** di $K+$ con $opad$ per produrre il blocco di b bit S_o .
6. Appendere il risultato dell'hash del passaggio 4 a S_o .
7. Applicare H allo stream generato nel passaggio 6 e produrre l'output.

Sicurezza di HMAC

La sicurezza di **HMAC** dipende direttamente dalla robustezza crittografica della funzione *hash* sottostante. Il grande valore di questo protocollo risiede nella capacità dei suoi progettisti di dimostrare una relazione esatta tra la forza della funzione *hash* integrata e quella di HMAC. In pratica, per un dato livello di sforzo applicato a messaggi generati da un utente legittimo e intercettati da un attaccante, la probabilità di riuscire a violare HMAC equivale a portare a termine con successo uno dei seguenti attacchi sulla funzione *hash*: calcolare un output della funzione di compressione nonostante l'impiego di un **IV (Vettore di Inizializzazione)** casuale, segreto e sconosciuto all'attaccante, oppure individuare delle collisioni nella funzione *hash* anche quando l'IV è mantenuto segreto e casuale.

Authenticated Encryption (AE)

La **Cifratura Autenticata (AE)** è un termine usato per descrivere sistemi di cifratura che proteggono simultaneamente la **riservatezza** e l'**autenticità** (integrità) delle comunicazioni. In altre parole, l'AE fornisce sia la cifratura del messaggio che la sua autenticazione. Molte applicazioni e protocolli richiedono entrambe le forme di sicurezza, ma fino a poco tempo fa i due servizi venivano progettati separatamente.

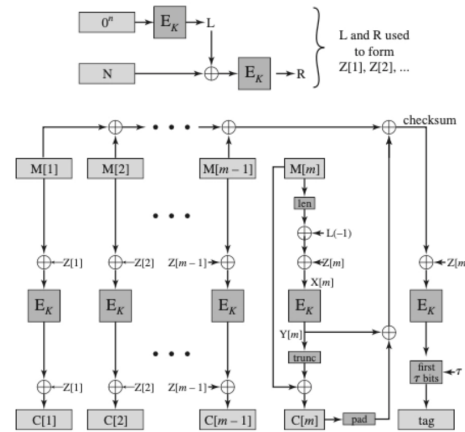
L'AE è implementata utilizzando una struttura a modalità di cifratura a blocchi. Un esempio utilizzato in numerose applicazioni è il **CCM**. In questa sezione, esaminiamo l'**Offset Codebook (OCB)**.

Caratteristiche dell'OCB

- **Standard:** Proposto dal NIST, è uno standard Internet definito nella **RFC 7253** ed è approvato nello standard wireless LAN IEEE 802.11. È incluso anche in *MiniSec*, un modulo di sicurezza IoT open-source.
- **Efficienza:** L'obiettivo chiave di OCB è l'efficienza. Questo viene ottenuto minimizzando il numero di cifrature richieste per messaggio e consentendo l'**operazione parallela** sui blocchi di un messaggio.
- **Singolo Passaggio:** È richiesto un solo passaggio sul messaggio per generare sia il testo cifrato (*ciphertext*) che il codice di autenticazione.
- **Configurazione Tipica:** Di solito si usa **AES** come algoritmo di cifratura con blocchi da $n=128$ bit.

Struttura e Funzionamento di OCB

La struttura di OCB è simile alla modalità *Electronic Codebook* (ECB), dove ogni blocco è cifrato indipendentemente. Tuttavia, l'ECB è insicuro perché blocchi identici producono testi cifrati identici. OCB elimina questo problema utilizzando un **offset $Z[i]$** unico per ogni blocco.



La formula fondamentale

Per ogni blocco $M[i]$, il testo cifrato $C[i]$ si ottiene come:

$$C[i] = E_K(M[i] \oplus Z[i]) \oplus Z[i]$$

Prima si applica lo XOR tra il blocco di testo in chiaro e l'offset $Z[i]$, poi si cifra il risultato con la chiave K , e infine si applica nuovamente lo XOR con lo stesso offset $Z[i]$. Grazie all'offset, due blocchi identici nella stessa posizione produrranno testi cifrati diversi perché i loro $Z[i]$ saranno differenti.

Il Nonce (N)

Viene scelto un valore a n bit chiamato **nonce**. L'unico requisito è che, se più messaggi vengono cifrati con la stessa chiave, deve essere utilizzato un *nonce* diverso ogni volta (il *nonce* viene usato solo una volta).

Calcolo degli Offset $Z[i]$

Il calcolo degli offset è la parte tecnicamente più complessa ed è riassunto dalle seguenti equazioni:

1. Valore iniziale L : $L(0) = L = E_K(0^n)$ (Cifratura di una stringa di zeri).
2. Valore R : $R = E_K(N \oplus L)$ (Cifratura del Nonce XORato con L).
3. Generazione dei sotto-valori $L(i)$: $L(i) = 2 \cdot L(i-1)$ per $1 \leq i \leq m$.
4. Primo Offset: $Z[1] = L \oplus R$.
5. Offset Successivi: $Z[i] = Z[i-1] \oplus L(\text{ntz}(i))$.

Nota Tecnica: L'operatore $\cdot$ si riferisce alla moltiplicazione nel campo finito $GF(2^n)$. L'operatore $\text{ntz}(i)$ indica il numero di zeri finali (meno significativi) in i . I valori $Z[i]$ risultanti mantengono una distanza di Hamming massima tra loro.

Gestione dell'Ultimo Blocco e del Tag

Poiché la lunghezza di M potrebbe non essere un multiplo intero di n , l'ultimo blocco viene trattato diversamente:

1. Si calcola $X[m]$:

$$X[m] = \text{len}(M[m]) \oplus L(-1) \oplus Z[m]$$

2. Si ottiene $T[m]$ tramite cifratura:

$$Y[m] = E_K(X[m])$$

3. $C[m]$ viene generato tramite XOR tra $M[m]$ e i primi bit necessari di $Y[m]$. Il testo cifrato finale C ha quindi la stessa lunghezza del testo in chiaro originale M .

Autenticazione (Il Checksum e il Tag)

Viene prodotto un **checksum** dal messaggio M seguendo questa formula:

$$\text{checksum} = M[1] \oplus M[2] \oplus \dots \oplus Y[m] \oplus C[m]0^*$$

Infine, viene generato un **tag di autenticazione** di lunghezza τ :

$$\text{tag} = \text{primi } \tau \text{ bit di } E_K(\text{checksum} \oplus Z[m])$$

Per verificare il messaggio, il ricevente ricalcola il *checksum* e il tag: se il tag calcolato corrisponde a quello ricevuto, il messaggio è autentico e integro.

Cifratura a Chiave Pubblica: RSA

L'algoritmo RSA, sviluppato nel 1977 da Ron Rivest, Adi Shamir e Len Adleman presso il MIT, rappresenta lo schema a chiave pubblica più ampiamente implementato. Si basa sulla teoria dei numeri, trattando il testo in chiaro e il testo cifrato come interi compresi tra 0 e $n-1$ per un determinato valore di n .

Descrizione dell'Algoritmo

Il sistema utilizza l'esponenziazione di interi in aritmetica modulare. Sia il mittente che il destinatario devono conoscere i valori di n ed e , che costituiscono la chiave pubblica. Solo il destinatario conosce il valore di d , che costituisce la chiave privata.

- **Chiave Pubblica (PU):** $\{e, n\}$.
- **Chiave Privata (PR):** $\{d, n\}$.

$$\text{Cifratura: } C \equiv M^e \pmod{n}$$

$$\text{Decifratura: } M \equiv C^d \pmod{n} \equiv (M^e)^d \pmod{n} \equiv M^{ed} \pmod{n}$$

Generazione delle Chiavi

Il processo di generazione delle chiavi segue passaggi matematici precisi per garantire la relazione tra le componenti:

1. **Selezione dei primi:** Scegliere due numeri primi distinti, p e q .
2. **Calcolo del modulo:** Calcolare $n = p \times q$.
3. **Funzione Totiente di Eulero:** Calcolare $\phi(n) = (p-1)(q-1)$. Questa funzione indica il numero di interi positivi minori di n e coprimi con esso.
4. **Selezione dell'esponente pubblico:** Scegliere un intero e tale che $\gcd(\phi(n), e) = 1$ e $1 < e < \phi(n)$.
5. **Calcolo dell'esponente privato:** Determinare d tale che $de \equiv 1 \pmod{\phi(n)}$, ovvero d deve essere l'inverso moltiplicativo di e modulo $\phi(n)$.

Requisiti di Sicurezza e Prestazioni

Perché il sistema sia soddisfacente, devono essere soddisfatti i seguenti requisiti:

- **Correttezza:** Deve essere possibile trovare valori di e , d , n tali che $M^{ed} \equiv M \pmod{n}$ per ogni $M < n$.
- **Efficienza:** Il calcolo di M^e e C^d deve essere relativamente semplice.
- **Inviolabilità:** Deve essere *computazionalmente* impossibile determinare d conoscendo solo e ed n . Questo requisito è soddisfatto utilizzando valori molto grandi per e e n .

Analisi degli Attacchi e Vulnerabilità

La sicurezza dell'RSA è costantemente valutata attraverso diversi approcci di attacco:

1. Attacchi Matematici (Fattorizzazione)

L'attacco matematico principale consiste nel tentare di fattorizzare n nei suoi fattori primi p e q . Se un attaccante riesce a fattorizzare n , può calcolare $\phi(n)$ e successivamente determinare d .

- **Benchmark di sicurezza:** Le prestazioni degli algoritmi di fattorizzazione fungono da benchmark per valutare la sicurezza dell'RSA.
- **Raccomandazioni:** Attualmente si consigliano chiavi di dimensione compresa tra 1024 e 2048 bit.

1. p e q devono differire in lunghezza solo di poche cifre.
2. Sia $(p - 1)$ che $(q - 1)$ devono contenere un fattore primo grande.
3. $\gcd(p - 1, q - 1)$ deve essere piccolo.

2. Attacchi Temporal (Timing Attacks)

Paul Kocher ha dimostrato che un osservatore può determinare una chiave privata monitorando il tempo impiegato da un computer per decifrare i messaggi. L'attacco analizza le variazioni temporali nell'esponentiazione modulare, che viene eseguita bit per bit.

Contromisure:

- **Tempo di esponentiazione costante:** Garantisce che tutte le operazioni richiedano lo stesso tempo, sebbene possa degradare le prestazioni.
- **Ritardo casuale:** Aggiunge un ritardo variabile per confondere l'analisi temporale.
- **Blinding (Oscuramento):** Moltiplica il testo cifrato per un numero casuale prima dell'esponentiazione:

1. Genera un numero casuale segreto r tra 0 e $n - 1$
2. Calcola $C' \equiv C(r^e) \pmod{n}$
3. Calcola $M' \equiv (C')^d \pmod{n}$
4. Calcola $M \equiv M'r^{-1} \pmod{n}$

Esempio Pratico dell'Algoritmo RSA

1. Generazione delle Chiavi

Scelta Primi: $p = 17, q = 11$
Calcolo Modulo: $n = p \times q = 187$
 $\phi(n) = (p - 1)(q - 1) = 160$
Scelta e : $\gcd(160, 7) = 1 \implies e = 7$
Calcolo d : $23 \times 7 \equiv 1 \pmod{160} \implies d = 23$
Chiave Pubblica: $PU = \{7, 187\}$
Chiave Privata: $PR = \{23, 187\}$

2. Operazioni (Messaggio $M = 88$)

Cifratura: $C \equiv 88^7 \pmod{187} = 11$
Decifratura: $M \equiv 11^{23} \pmod{187} = 88$

Cifratura a Chiave Pubblica: Lo scambio di chiavi Diffie-Hellman

L'algoritmo Diffie-Hellman, pubblicato nel 1976, è stato il primo protocollo a chiave pubblica a comparire nella letteratura scientifica. A differenza dell'RSA, il suo scopo primario non è la cifratura generica di messaggi, ma consentire a due utenti di scambiarsi in modo sicuro una **chiave segreta** che potrà poi essere utilizzata per la successiva cifratura dei messaggi tramite algoritmi simmetrici.

L'efficacia del protocollo si basa sulla difficoltà computazionale del calcolo dei **logaritmi discreti**.

- **Radice Primitiva:** Si definisce radice primitiva α di un numero primo q quel valore le cui potenze generano tutti gli interi distinti da 1 a $q-1$ modulo q .
- **Logaritmo Discreto:** Data l'equazione $b \equiv a^i \pmod{p}$, è relativamente facile calcolare l'esponente i (il logaritmo discreto) conoscendo b , a e p . Per numeri primi di grandi dimensioni, questo compito è considerato impossibile.

Dati q (numero primo) e α (radice primitiva di q):

$$Y \equiv \alpha^X \pmod{q}$$

$$X = \text{dlog}_{\alpha, q}(Y) \quad (\text{Problema difficile})$$

Descrizione dell'Algoritmo

Il protocollo prevede la condivisione di elementi pubblici globali e la generazione di chiavi private e pubbliche per ciascun utente.

1. Elementi Pubblici Globali:

Un numero primo q e una radice primitiva α di q .

2. Generazione delle Chiavi Utente:

- **Utente A:** Seleziona una chiave privata $X_A < q$ e calcola la chiave pubblica $Y_A \equiv \alpha^{X_A} \pmod{q}$
- **Utente B:** Seleziona una chiave privata $X_B < q$ e calcola la chiave pubblica $Y_B \equiv \alpha^{X_B} \pmod{q}$

3. Calcolo della Chiave Segreta Condivisa (K):

- L'utente A calcola: $K \equiv (Y_B)^{X_A} \pmod{q}$
- L'utente B calcola: $K \equiv (Y_A)^{X_B} \pmod{q}$
- Entrambi ottengono lo stesso risultato grazie alle proprietà delle potenze.